Lecture 4: Stochastic Thinking and Random Walks

This lecture opens a new unit by arguing that randomness is not a confession of ignorance but a legitimate scientific stance — physics itself may be nondeterministic, and even where it is not, our own ignorance forces us to model the world as unpredictable — and then turns that stance into runnable code.

Why embrace randomness?

The lecture begins with a tension worth sitting with: **uncertainty is uncomfortable** — people want answers and predictions — but **certainty is usually unjustified**. That discomfort pushes us to claim more confidence than we deserve, and much of the course is about reasoning quantitatively in the honest middle ground where "probably" replaces "definitely."

Computationally, this means shifting away from programs that do exactly what we tell them toward programs and models that *embrace* randomness. The Wikipedia record shows why this is powerful rather than lazy: for centuries randomness was treated as an obstacle and nuisance, but in the twentieth century computer scientists realized that **deliberately introducing randomness into computations can be an effective tool**, and in some cases **randomized algorithms even outperform the best deterministic methods**. Monte Carlo methods, which rely on random input from random number generators (or pseudorandom number generators), became important techniques across computational science. The key that makes this respectable: individual random events are unpredictable, but with a known probability distribution the *frequency* of outcomes over repeated trials is predictable.

The Newtonian clockwork universe

The dominant pre-twentieth-century picture rests on two pillars:

1. **Every effect has a cause** — nothing just happens.
2. **The world can be understood causally** — if you know the state of a system and the forces acting on it, you can in principle compute exactly what happens next.

This is a deterministic, predictable, comfortable "clockwork universe." Its importance here is as the foil: under this worldview, randomness could only mean *our* ignorance, never a property of the world.

The Copenhagen doctrine: causal nondeterminism

Twentieth-century physics undermined that comfort. What the lecture calls the **Copenhagen Doctrine**, proposed by Bohr and Heisenberg (Wikipedia adds Max Born and others), asserts **causal nondeterminism**: *at its most fundamental level, the behavior of the physical world cannot be predicted.* Not "we don't know enough yet" — fundamentally unpredictable. Under this view, statements of the form "**x is highly likely to occur**" are perfectly good science, but statements of the form "**x is certain to occur**" are off the table, even in principle.

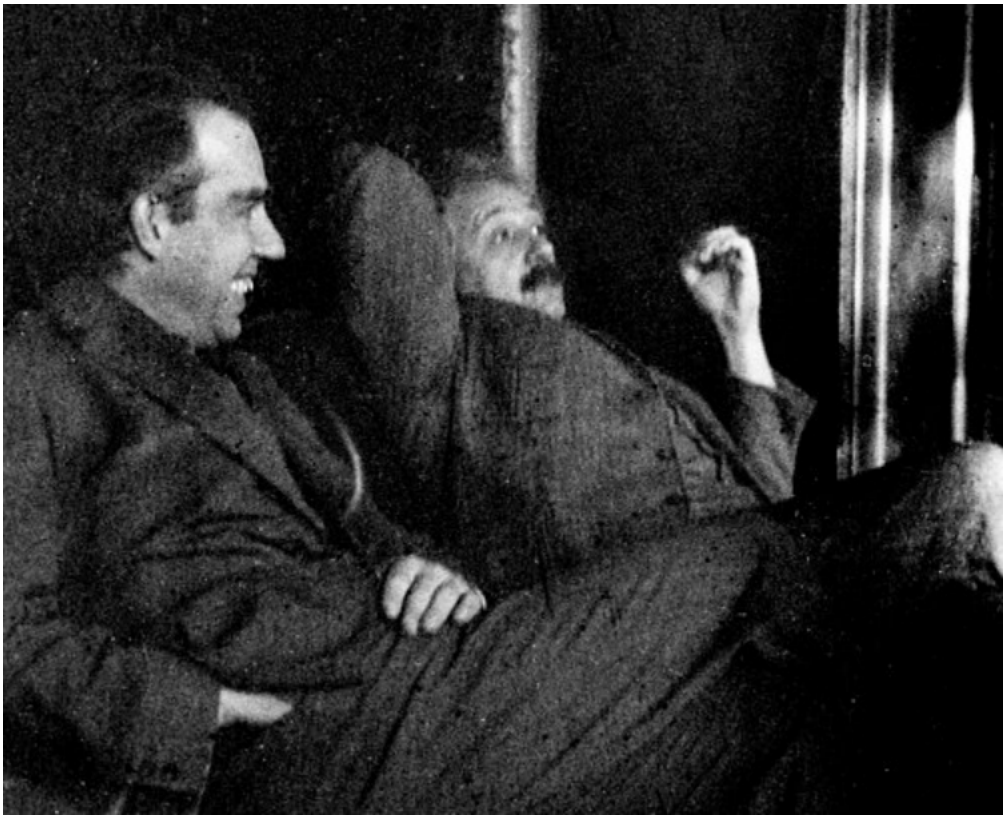
The Wikipedia account adds crucial texture. There is no definitive historical statement of the interpretation — the name was apparently coined by Heisenberg around 1955 (partly to criticize alternatives such as David Bohm's), referring back to ideas developed in 1925–1927, and Bohr's and Heisenberg's writings contradict each other on important issues. Features common across versions include:

- Quantum mechanics is **intrinsically indeterministic**, with probabilities calculated using the **Born rule**;
- **Complementarity**: objects have pairs of complementary properties that cannot all be observed or measured simultaneously;
- The act of measurement is **irreversible**, and no truth can be attributed to an object except according to measurement results (rejection of counterfactual definiteness);
- Yet quantum descriptions are **objective**, independent of physicists' personal beliefs.

Historically, this grew out of a quarter-century of crisis: starting in 1900, atomic and subatomic phenomena forced revisions to classical physics (Planck's blackbody spectrum, Einstein's photoelectric effect, Bohr's

hydrogen atom), and after the heuristic "old quantum theory" stalled on helium, Heisenberg's 1925 treatment built only on observable quantities, Born recast position and momentum as matrices and interpreted Schrödinger's wave function as a *tool for calculating probabilities*, and at the 1927 Solvay Conference Born and Heisenberg declared quantum mechanics a closed theory. Objections persist — the discontinuous, stochastic nature of measurement, the difficulty of defining a measuring device, its reliance on classical physics — yet it remains one of the most commonly taught interpretations.

Not everyone bought it: **Einstein and Schrödinger** objected strenuously, Einstein summarizing his position as "**God does not play dice**." That objection has a living intellectual lineage — hidden-variable theories exist precisely to reject the idea that nature contains irreducible randomness. But under the standard interpretations, microscopic phenomena are objectively random: an unstable atom in a controlled environment has an unpredictable decay time, with only a probability of decay in a given interval; quantum mechanics specifies probabilities, not individual outcomes.



Source: Wikipedia, article "Copenhagen interpretation".

Predictive nondeterminism: treat the world as random

Flip a coin twice. Did you get two heads, two tails, or one of each? Before looking, there is no way to know — and, crucially, *it doesn't matter whether the coin is fundamentally deterministic*: from where we sit, the outcome is effectively unpredictable. Hence the moral of the story: the world may or may not be inherently unpredictable, but **our lack of knowledge does not allow us to make accurate predictions**, so we might as well treat the world as inherently unpredictable. The lecture names this principle **predictive nondeterminism**: it does not claim the world *is* random; it claims that for all practical purposes we should *model* it as if it were — a perfectly respectable scientific stance.

Two ideas from the Wikipedia material show why the stance holds up. First, even classically deterministic systems can be effectively unpredictable: the ball in a roulette wheel behaves in a way that is **very sensitive**

to initial conditions, making it a source of apparent randomness. Second, randomness is better understood **not as haphazardness but as a measure of uncertainty of an outcome**: individual events are unpredictable by definition, yet frequencies follow known distributions — throw two dice and any particular roll is unpredictable, but a sum of 7 will tend to occur twice as often as 4. Treating the world as random therefore doesn't abandon prediction; it relocates prediction from individual outcomes to distributions.

Stochastic processes

To formalize this, the lecture introduces the **stochastic process**: an ongoing process where the next state might depend on both the previous states *and some random element*. The system can have history and structure — the new ingredient is the random component.

The formal version grounds this precisely: a stochastic process is a **family of random variables in a probability space**, indexed by a set that usually carries the interpretation of time. Each random variable takes values in a shared **state space** (integers, the real line, or n-dimensional Euclidean space); an **increment** is the amount the process changes between two index values; and a single outcome of the process is called a **sample function** or **realization**. A random variable itself is an assignment of a numerical value to each possible outcome of an event space, which is what makes probabilities calculable. Classification matters: if the index set is finite or countable the process is in **discrete time** (easier to study; integer-indexed processes are called random sequences), whereas an interval of the real line gives **continuous time**. The taxonomy includes random walks, martingales, Markov processes, Lévy processes, Gaussian processes, renewal processes, and branching processes — with **random walks**, the subject of this unit, heading the list. The two classic examples are the Wiener process (Brownian motion), which Louis Bachelier used to model price changes on the Paris Bourse, and the Poisson process, which A. K. Erlang used to model phone calls; applications now span biology, chemistry, ecology, neuroscience, physics, signal and image processing, control and information theory, computer science, telecommunications, and finance.

Implementing a random process in Python

A specification exercise makes the definition concrete. Consider two specs:

```
def rollDie():
    """Returns an int between 1 and 6."""
```

versus

```
def rollDie():
    """Returns a randomly chosen int between 1 and 6."""
```

The first is satisfied by a function that *always returns 3* — it says nothing about randomness. The single added word "randomly" is the entire difference: it supplies the random element of the stochastic process.

The implementation uses Python's `random` module:

```
import random

def rollDie():
    """Returns a random int between 1 and 6."""
    return random.choice([1, 2, 3, 4, 5, 6])

def testRoll(n = 10):
    result = ''
    for i in range(n):
        result = result + str(rollDie())
    print(result)
```

Each call to `rollDie` picks **uniformly** one of the six faces — and this matches the formal requirement for random selection: a random selection mechanism requires **equal probabilities for any item to be**

chosen, here 1/6 per face. The harness `testRoll` concatenates n rolls into a digit string (ten by default), and each run prints something different — the randomness made visible.

One caution from the Wikipedia data on random sampling: random does **not** mean proportionally representative. Drawing 10 marbles from a bowl of 10 red and 90 blue marbles need not yield exactly 1 red and 9 blue, even though each draw picks red with probability 1/10. Small samples routinely fail to look "typical."

The open question: how probable is 11111?

That caution sets up the lecture's closing puzzle. Run `testRoll(5)` and ask: how probable is the output `11111`? Five ones in a row *looks* suspicious — far less likely, intuitively, than something like `53416`. But is it actually less likely, and how would you even go about computing a quantity like $P(\text{output} = \texttt{11111})$? The marble-bowl lesson warns against trusting the intuition that a patterned string must be rare, while the distribution

When Simulation Fails: Estimating the Probability of a Rare Event

A Monte Carlo simulation ran against a rare event produced a striking result: the **actual** probability was 0.0001286 — a little more than one occurrence per ten thousand trials — while the **estimated** probability printed as 0.0, reproducibly across two runs. The simulation declared the event essentially impossible when it in fact happens slightly more than once in ten thousand.

This outcome was predictable in advance, and the mechanism is worth internalizing. An estimator built from random sampling computes hits divided by trials. When the true probability p is tiny, a typical trial almost never produces the event, so with any modest number of trials n the hit count is zero, and $0/n = 0$. Formally, the probability of seeing zero hits in n independent trials is $(1-p)^n \approx e^{-np}$: unless n is enormous compared to $1/p \approx 7776$, zero hits is the *likely* outcome, not a bug. Even at $n = 10{,}000$ trials the expected number of hits is only $np \approx 1.29$, so an empty result remains quite plausible. Knowing only that p is very small therefore lets you predict the printout before running the code.

This is a recognized limitation of the Monte Carlo approach generally: such methods obtain numerical results by repeated random sampling, their justification resting on the law of large numbers — the empirical (sample) mean of independent samples converges to the true expected value — and their practicality involves a trade-off between accuracy and computational cost. Convergence is not free, and for rare events it is expensive. The proposed remedy in lecture was brute force: throw 1,000,000 trials at the problem and see whether the estimate starts to resemble the truth.

Three Morals About Simulation

The failed rare-event experiment generalizes into three lessons:

1. **It takes a lot of trials to get a good estimate of the frequency of a rare event.** How many trials are *enough* is a question you should ask yourself every time you run a simulation; tools for answering it come in later lectures. This is the accuracy-versus-cost trade-off inherent to random-sampling methods.
2. **Do not confuse the sample probability with the actual probability.** A simulation yields an estimate constructed from random samples, and that estimate is not the truth — as demonstrated, it can be off by a great deal (0.0 versus 0.0001286).
3. **If a closed-form answer exists, use it.** This particular problem had a perfectly good analytic solution, so simulating it was unnecessary. Many upcoming examples will have *no* closed form — those are where simulation earns its keep, providing approximate solutions to problems too complex for mathematical analysis. The closing caveat: simulations are nonetheless often useful, which is exactly where the lecture goes next.

The Birthday Problem

The famous example is the **birthday problem**: what is the probability that at least two people in a group share a birthday? Two anchor points frame it. With 367 people the answer is trivially 1: counting February

29th there are only 366 possible birth dates, so a collision is guaranteed. The interesting regime is smaller groups.

Under the simplifying assumption that every birth date is equally likely, there is a closed-form answer. Count the ways all N birthdays could be distinct and subtract from one:

$$P(\text{at least two share}) \;=\; 1 - \frac{366!}{366^N\,(366-N)!}$$

The numerator's logic: the first person can have any of 366 dates, the second any of the remaining 365, and so on, giving $366 \cdot 365 \cdots (366 - N + 1) = \frac{366!}{(366-N)!}$ all-distinct assignments out of 366^N total assignments. Equivalently, via conditional probability, person 2 avoids person 1's date with probability $\frac{365}{366}$, person 3 avoids both with probability $\frac{364}{366}$, and these multiply.

Without the uniformity assumption the problem gets *very* complicated — and real birthdays are not uniformly distributed. The theory says a uniform distribution actually *minimizes* the probability of a shared birthday: any unevenness in birth rates increases the likelihood of a match. Fortunately, real-world birthdate variation is not sufficiently uneven to matter much — the group size needed for a greater-than-50% chance of a match stays at 23, matching the theoretical uniform model.

That 23-person threshold is the celebrated **birthday paradox**: a *veridical* paradox — it seems wrong at first glance but is in fact true. With 365 equally likely days, at $k = 23$ people the all-distinct count is $V_{nr} = \frac{365!}{(365-23)!}$ against $V_t = 365^{23}$ total arrangements, so

$$P(A) = \frac{V_{nr}}{V_t} \approx 0.492703, \qquad P(B) = 1 - P(A) \approx 0.507297,$$

i.e., a shared birthday is already more likely than not with only 23 people — fewer than 1/15th of the days in a year. The intuition repair is to count *pairs*: with 23 people there are $\frac{23\times 22}{2} = 253$ pairwise comparisons, any one of which can produce the match. The problem is generally attributed to Harold Davenport around 1927 (unpublished; he didn't claim discovery because he couldn't believe it hadn't been stated earlier), with the first publication by Richard von Mises in 1939. Beyond party trivia it matters practically: the **birthday attack** in cryptography exploits this probabilistic model to reduce the complexity of finding collisions in hash functions.

Simulating the Birthday Problem

The simulation replaces the formula with direct sampling, in two layers.

One trial — `sameDate(numPeople, numSame):` - Set `possibleDates` to `range(366)` — the 366 possible days. - Initialize `birthdays` as a list of 366 zeros: one counter per possible date. - For each of the `numPeople` people, draw a `birthDate` with `random.choice(possibleDates)` and increment `birthdays[birthDate]`. - Return `max(birthdays) >= numSame` — True exactly when some single date was hit at least `numSame` times. With `numSame = 2`, this is True precisely when at least two people in the simulated group share a birthday.

Many trials — `birthdayProb(numPeople, numSame, numTrials):` - Initialize `numHits` to 0; for each trial call `sameDate(numPeople, numSame)` and increment `numHits` on True. - Return `numHits / numTrials` — the sample probability, our estimate.

Running this for `numPeople` in $\{10, 20, 40, 100\}$ with `numSame = 2` and `numTrials = 10000`, alongside the exact closed-form value (computed as `1 - numerator/denom` with `numerator = math.factorial(366)` and `denom = 366**numPeople * math.factorial(366 - numPeople)`, checked at $N = 100$), shows the estimates tracking the actual probabilities closely — with enough trials, simulation gets us quite close to the truth, vindicating the method once the rare-event pitfall is respected.

The payoff appears when the question changes to **three** people sharing a birthday. Analytically, the tidy "all-distinct count subtracted from one" no longer describes the event, and the closed form becomes much

messier. In the simulation, it is essentially a one-character change: pass `numSame = 3` instead of `2`. This is Moral 3 operating from the other side — when no easy closed form exists, simulation is often the cheap way in.